Understanding Foundation Models

Training data, architecture, scaling, alignment, sampling, and probabilistic behavior

A model's behavior emerges from what it was trained on, how it was built, what post-training optimized, and how generation is sampled. All four matter when selecting and debugging a model.

Source: Chip Huyen, AI Engineering: Building Applications with Foundation Models, first edition (2025). Source: book pp. 49–112 / PDF p. 73. These are original study notes, not verbatim chapter reproductions. Tool names, prices, model comparisons, and research findings reflect the book's publication period unless stated otherwise.

01 Training Data

Source: book pp. 50–58 / PDF p. 74

Web data, multilingual models, and representation

Broad web corpora provide scale but contain misinformation, duplicates, uneven representation, and content with uncertain rights. Availability does not imply suitability. Models inherit capabilities and biases from these distributions.

High-resource languages receive more training exposure than low-resource languages, often disproportionately to speaker populations. A model strong in English may be much worse at reasoning, safety, and instruction-following in another language. Tokenization can make the same meaning cost more tokens and consume more context in underrepresented languages. Evaluate realistic language and dialect inputs, not translated benchmarks alone.

Domain-specific models

General-purpose corpora include many domains, but specialist tasks may require representations and data absent from the web: molecular sequences, clinical images, proprietary schemas, or niche terminology. Check the actual task rather than assuming a domain-branded model is superior.

Training distribution can explain why two otherwise similar models differ on specialized image or text datasets. Domain coverage, data quality, task objectives, and post-training all matter; parameter count alone is an incomplete model description.

02 Modeling: Architecture and Size

Source: book pp. 58–78 / PDF p. 82

Transformer architecture and attention

Older encoder–decoder systems compressed inputs into a recurrent state and processed sequences sequentially. Attention lets the model directly weight relevant token representations; transformers allow substantial parallel processing during training and prompt processing. Autoregressive output generation remains sequential unless specialized techniques change it.

$$Q = XW_Q,\ K = XW_K,\ V = XW_V$$
$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

Queries determine what a position seeks, keys support matching, and values supply the information combined. Multi-head attention captures different relationships. Causal masking prevents using future tokens for next-token prediction.

Where parameters live and alternative architectures

Transformer blocks combine attention and feedforward networks, with nonlinearities and supporting normalization/residual operations. Hidden dimensions, layer count, feedforward dimensions, vocabulary embeddings, and output projections determine parameter count. Larger context increases runtime memory and computation without necessarily adding weight parameters.

Alternatives include state-space approaches such as S4/Mamba and hybrids such as Jamba. Mixture-of-experts models activate selected experts per token: total parameters and active parameters are different quantities. Architecture affects hardware behavior, adaptation options, and serving cost, so dense-model comparisons do not transfer mechanically to sparse/hybrid models.

Parameters, tokens, FLOPs, and compute budgets

Model size commonly means parameter count, not the byte size of a stored checkpoint. Weight memory also depends on numerical precision. Dataset size is often measured in tokens because documents and examples vary enormously in length.

$$\text{Weight storage} \approx \text{parameter count} \times \text{bytes per parameter}$$
$$\text{Dense-transformer training compute} \approx 6ND \text{ FLOPs}$$
$$N = \text{parameters};\ D = \text{training tokens}$$

The compute expression is a rough dense-model estimate, not a universal law. FLOPs count operations; FLOP/s is a rate. Runtime depends on achieved throughput and utilization, not advertised peak performance alone. Training budgets also include data, experiments, hardware availability, and engineering.

Scaling laws and compute-optimal training

Performance depends jointly on model capacity, data, and compute. Compute-optimal scaling allocates a fixed budget between parameters and tokens rather than making only the model larger. The Chinchilla result is often summarized as roughly 20 training tokens per parameter in its studied setting, with model and token counts growing together.

This is a training-compute optimum under assumptions, not necessarily a lifetime-cost optimum: a heavily used small model may justify more pre-training because inference savings dominate. Small loss improvements can unlock meaningful capabilities, but gains generally diminish. Extrapolate from smaller experiments cautiously because hyperparameters and data mixes interact. Inverse scaling reminds us that not every measured behavior improves monotonically with size.

Scaling bottlenecks

Compute, energy, high-quality data, and practical infrastructure limit scaling. Public data can be exhausted, contaminated, or intentionally manipulated. Increasingly synthetic web data creates provenance and recursion risks. More data of the wrong kind is not a substitute for relevant signal.

Parameter versus hyperparameter: weights are learned; choices such as architecture size, learning rate, batch size, and training schedule govern learning. They must be tuned and recorded to make scaling experiments interpretable.

---

03 Post-Training

Source: book pp. 78-88 / PDF p. 102

Supervised fine-tuning (SFT)

A pre-trained model can complete arbitrary web-like text without behaving like a helpful assistant. SFT uses demonstration pairs of prompts and desired responses to teach useful response patterns, tool formats, styles, and conversational behavior. It is sometimes described as behavior cloning.

Good demonstrations require expertise, judgment, consistency, and coverage; sophisticated answers are expensive to annotate. Post-training may expose existing capabilities while teaching new behavior, but it does not guarantee truthfulness or safety. Pre-training, SFT, and preference optimization are a common recipe, not mandatory fixed stages for every model.

Preference fine-tuning, reward models, RLHF, DPO

Demonstrations do not fully specify which of many plausible answers people prefer. Comparison data records a prompt, winning response, and losing response. A reward model can learn to score preferred responses more highly.

> Pairwise reward loss = $-\log \sigma(r(x,y_{win}) - r(x,y_{lose}))$

In RLHF, a policy generates answers and is optimized using reward-model feedback, often with PPO and a constraint against drifting too far from a reference. DPO optimizes preferences directly without the separate online RL loop. Neither captures a universal human preference: annotator disagreement, reward hacking, excessive refusal, and sycophancy are real risks. Evaluate the intended audience and application.

---

04 Sampling Fundamentals and Strategies

Source: book pp. 88-96 / PDF p. 112

Logits, probabilities, and temperature

Each decoding step produces logits over the vocabulary. Softmax converts them into probabilities; sampling chooses the next token. Log probabilities avoid underflow and turn sequence-probability products into sums.

> $p_i = \exp(z_i/T) / \Sigma_j \exp(z_j/T)$

Lower temperature concentrates probability on common choices; higher temperature flattens the distribution. A provider's "temperature zero" usually means greedy selection, not literal division by zero. Temperature does not add knowledge and cannot guarantee factuality.

Top-k, top-p, and stopping

Top-k limits candidates to the k highest-scoring tokens. **Top-p/nucleus** uses the smallest high-probability set whose cumulative probability reaches the chosen threshold. **Min-p** uses a threshold relative to the most likely token. These settings affect diversity and interact with temperature.

Stop at an end token, configured stop sequence, or token limit. Too-short limits can truncate JSON, reasoning, or answers; loose limits add cost and may encourage rambling. Record generation settings with the model and prompt so experiments can be reproduced.

---

05 Test-Time Compute and Structured Outputs

Source: book pp. 96-105 / PDF p. 120

Best-of-N, self-consistency, and verifiers

Spend more inference compute by generating multiple candidates, verifying them, reranking them, or aggregating answers. Best-of-N needs a useful scoring rule. Self-consistency uses agreement across samples, which can still be consistently wrong. Average log probability may prefer fluent familiar answers over correct uncommon ones.

> $\log p(x_{1:n}) = \Sigma_i \log p(x_i \mid x_{<i})$

More samples improve opportunity, not guaranteed final accuracy: weak verifiers can select convincing errors. Parallel candidate generation can sometimes return the first validated answer sooner, but uses more resources. Match the strategy to your latency and budget.

Getting outputs into the right format

Approaches include clear prompts/examples, repair through post-processing, retries/test-time compute, constrained decoding, and fine-tuning. Constrained sampling masks tokens that violate a grammar or schema. A classifier head can replace free-form generation for some fixed-label tasks.

Syntax is not semantics: valid JSON is not necessarily schema-valid, and schema-valid content is not necessarily true or authorized. Validate field types, allowed values, cross-field relationships, and business rules. Repair and retries need explicit limits. Constraints may increase latency or change generation quality, so measure them end to end.

06

Probabilistic Behavior: Inconsistency and Hallucination

Source: book pp. 105-112 / PDF p. 129

Inconsistency has several sources

Identical prompts can produce different answers; tiny prompt changes can also cause large shifts. Sampling controls only some variability. Model versions, parallel computation, numerical precision, and hardware can affect results even under apparently deterministic settings.

Test prompt perturbations and repeated runs. Freeze accessible versions, record configurations, and detect drift. Build validation and fallback behavior rather than promising deterministic behavior that the deployment cannot support.

Why hallucinations can persist

The book describes complementary hypotheses. **Self-delusion:** a model conditions on its own mistaken earlier output as if it were evidence, compounding the error. **Knowledge mismatch:** supervised responses can train confident answering beyond what the model internally knows, instead of calibrated uncertainty.

Mitigations include evidence retrieval, independent verification, shorter grounded outputs, appropriate abstention, and training that rewards factual behavior. RLHF does not automatically remove hallucinations and can worsen some measures. Detecting hallucinations is itself difficult; use task-specific checks rather than relying solely on the model's self-reported confidence.

Revision Check

1. How do logits, temperature, top-k, and top-p differ?
2. Why can two equal-sized models have different capabilities?
3. What assumptions underlie compute-optimal scaling?
4. How does SFT differ from RLHF and DPO?
5. Why does valid structured output not guarantee a safe or correct action?

Treat model claims as hypotheses to test in your workload. Architecture, data, post-training, and decoding settings explain different parts of observed behavior.

Book-based notes, separate from the earlier transcript-based notes. Coverage is indexed by section in coverage.md. Source links open the supplied book at the relevant PDF page. Detailed examples, citations, and complete experimental evidence remain in the book.